# EGH450 AUT / Guidance Architecture Handover

## Purpose

This document gives the teaching team a concise overview of the current **Autopilot, Guidance and Navigation (AUT)** software architecture used by the EGH450 UAS.

It is intended to make the codebase easy to understand at a surface level during the demonstration, particularly if a small fault is intentionally introduced for troubleshooting.

The current architecture separates:

- **Vehicle/control infrastructure** — MAVROS, SPAR and payload interfaces.
- **Mission guidance** — autonomous mission logic and waypoint progression.
- **Route planning** — Breadcrumb obstacle-aware path generation.
- **Environment inputs** — OptiTrack and occupancy grid.

## 1. High-Level Architecture



The core design principle is:

```
Guidance decides WHERE to go.
Breadcrumb decides HOW to get there safely.
SPAR executes the requested movement.
MAVROS provides the interface to the flight controller.
```

## 2. Launch Files

### catkin_ws_G6/launch/control.launch

**Purpose:** starts the lower-level aircraft/control infrastructure.

```
control.launch
├── SPAR
├── MAVROS
├── mocap/pose relay
└── payload deployment subsystem
```

It should **not** start Guidance or Breadcrumb.

### catkin_ws_G6/src/spar/spar_node/launch/guidance.launch

**Purpose:** starts the autonomous mission layer.

```
guidance.launch
├── Breadcrumb
└── Guidance
```

It also loads:

```
spar_node/config/guidance.yaml
```

into the Guidance node.

### catkin_ws_G6/src/qutas_lab_450/launch/environment.launch

**Purpose:** provides the lab/environment information required by AUT.

Important outputs include:

```
/grid
```

for Breadcrumb planning, plus the OptiTrack pose source used by MAVROS/Guidance.

## 3. Main AUT Files

### Guidance node

```
catkin_ws_G6/src/spar/spar_node/scripts/guidance
```

This is the main autonomous mission controller. It owns:

- mission state progression;
- search waypoint progression;
- ROI/target handling;
- payload diversion;
- return-to-search behaviour;
- final ArUco landing approach;
- low-battery handling;
- Breadcrumb route requests;
- SPAR action calls;
- movement result/error handling.

The older separate `takeoff`, `land` and `demo_wp_roi` mission logic was consolidated into this single Guidance node.

### Guidance configuration

```
catkin_ws_G6/src/spar/spar_node/config/guidance.yaml
```

This contains the main tunable mission parameters:

```
flight:      heights, speeds, tolerances
roi:         payload approach settings and pauses
topics:      ROS topic names
spar:        SPAR action interface
navigation:  Breadcrumb service and sparse-path settings
mission:     ROI enable timing and lawnmower waypoints
battery:     warning/landing thresholds
```

The current lawnmower pattern uses intermediate waypoints so Breadcrumb is encouraged to return to the intended search line after avoiding an obstacle.

Typical horizontal points are:

```
-3.6 → -2.0 → 0.0 → +2.0 → +3.2
```

or the reverse direction.

## 4. Guidance Mission State Flow

Conceptually:

```
WAITING_FOR_FLIGHT
        │
        ▼
      TAKEOFF
        │
        ▼
       SEARCH
        │
        ├── normal waypoint progression
        │
        ├── ROI detected
        │       │
        │       ▼
        │  DIVERT_TO_TARGET
        │       │
        │       ▼
        │  DEPLOY_PAYLOAD
        │       │
        │       ▼
        │  RETURN_TO_SEARCH
        │
        ▼
 FINAL_APPROACH
        │
        ▼
     LANDING
        │
        ▼
    COMPLETE
```

An unrecoverable command or route failure can transition the mission to an aborted/failure state.

## 5. Breadcrumb Route Planning

### Breadcrumb package

```
catkin_ws_G6/src/breadcrumb/
```

Breadcrumb provides:

```
/breadcrumb/request_path
```

Guidance sends a start position and destination. Breadcrumb returns a full path and, when available, a sparse path.

Guidance prefers the sparse path.

### Current Breadcrumb behaviour

Breadcrumb:

- uses `/grid` as its occupancy map;
- respects configured mission bounds;
- blocks occupied cells;
- inflates internal obstacles by approximately **0.4 m** for additional clearance;
- avoids unnecessarily inflating the outer map boundary;

- snaps unsafe start/end points to a nearby free cell when required;
- supports diagonal/Theta* movement for general navigation;
- uses cardinal/rectangular movement for axis-aligned lawnmower requests;
- returns sparse corner points where possible.

The route-planning intent is:

```
normal navigation:
    direct path where safe

obstacle encountered:
    leave nominal line
    route around inflated obstacle
    reach next predefined search point
    continue search
```

The intermediate lawnmower waypoints make the intended search geometry explicit and improve path rejoining after a detour.

## 6. Guidance-to-Breadcrumb Interface

The main Guidance functions are:

```
plan_path()
```

Requests a safe path from Breadcrumb.

```
navigate_to()
```

Executes each returned Breadcrumb point through SPAR.

```
destination
    |
    ▼
plan_path()
    |
    ▼
Breadcrumb service
    |
    ▼
safe route points
    |
    ▼
navigate_to()
    |
    ▼
fly_to()
    |
    ▼
SPAR
```

If Breadcrumb fails to return a valid route, Guidance reports failure rather than falling back to unsafe direct movement.

## 7. SPAR and MAVROS

SPAR remains the lower-level movement/action interface.

Guidance sends SPAR target data such as:

```
X
Y
Z
yaw
movement speed
```

SPAR converts the request into the appropriate MAVROS setpoint behaviour.

```
Guidance
   |
   ▼
SPAR action server
   |
   ▼
MAVROS setpoints
   |
   ▼
Pixhawk
```

Guidance does not directly command the flight controller.

## 8. Heading Behaviour

Breadcrumb determines route geometry, but Guidance retains control of the commanded mission yaw.

Breadcrumb path orientation is ignored as the aircraft heading command.

The current search mission uses approximately:

```
yaw = 0.0 rad
```

so the UAV can translate around obstacles without intentionally yawing to face each segment.

## 9. ROI / Target Diversion Flow

Target types include:

```
A = ArUco landing marker
B = backpack
P = person
```

`B` and `P` can interrupt the search mission.

Typical diversion:

```
search waypoint
      |
target detected
      |
      ▼
store interruption position
      |
      ▼
Breadcrumb horizontal route to target
      |
      ▼
capture actual safe approach position
      |
      ▼
vertical payload descent
      |
      ▼
payload deployment
      |
      ▼
vertical climb
      |
      ▼
Breadcrumb return to interruption position
      |
      ▼
resume search waypoint
```

The actual safe XY approach position is captured before descent so the UAV does not route safely around an obstacle and then descend back toward an unsafe original target coordinate.

## 10. ArUco Landing Handling

An `A` ROI stores the ArUco landing location but does not immediately interrupt the search.

At the end of the search:

```
FINAL_APPROACH
      |
      ▼
Breadcrumb route to stored ArUco location
      |
      ▼
LANDING
```

If no ArUco location has been stored, Guidance uses its configured fallback landing behaviour.

## 11. Startup Interlocks and Pilot Authority

Guidance waits for required infrastructure before progressing.

It first checks for:

```
/breadcrumb/request_path
```

Breadcrumb only advertises this service after receiving a valid occupancy grid.

```
/grid available
      |
      ▼
Breadcrumb ready
      |
      ▼
Guidance allowed to continue
```

Guidance also waits for the SPAR action server and a valid UAV pose.

Pilot authority is preserved:

```
arming   = manual
OFFBOARD = manual
```

No AUT script automatically arms the aircraft or changes flight mode.

## 12. Motion Result Handling

Guidance uses explicit movement results rather than assuming success.

Important results include:

```
SUCCEEDED
FAILED
ROI_INTERRUPT
LOW_BATTERY
SHUTDOWN
```

Each SPAR command result is checked before mission progression continues.

## 13. Recommended Launch Order

Typical demonstration startup:

```
Terminal 1:
environment.launch

Terminal 2:
control.launch

Terminal 3:
guidance.launch
```

The pilot then manually:

```
ARM
select OFFBOARD
```

Guidance waits if the planner, SPAR server or pose data is unavailable.

## 14. Useful ROS Troubleshooting Checks

### Nodes

```
rosnode list
```

Important nodes should include approximately:

```
/guidance
/breadcrumb
/spar
/mavros
/area_map
```

### Breadcrumb service

```
rosservice list | grep breadcrumb
```

Expected:

```
/breadcrumb/request_path
```

If missing, check `/grid` first.

### Occupancy grid

```
rostopic hz /grid
```

### UAV pose

```
rostopic echo /mavros/local_position/pose
```

### MAVROS state

```
rostopic echo /mavros/state
```

Useful fields include connection state, armed state and flight mode.

### ROI enable

```
rostopic echo /guidance/roi_enable
```

### Parameters

```
rosparam get /breadcrumb/allow_diagonals
rosparam get /breadcrumb/any_angle
rosparam get /guidance/flight
```

## 15. Suitable Minor Fault-Injection Locations

These are relatively easy areas for a teaching-team troubleshooting fault without requiring major code changes.

### A. Incorrect Breadcrumb service name

Expected configuration:

```
/breadcrumb/request_path
```

Likely symptom: Guidance repeatedly waits for Breadcrumb.

Check:

```
rosservice list | grep breadcrumb
```

### B. Incorrect Breadcrumb grid remap

Expected:

```
~grid → /grid
```

Likely symptom: Breadcrumb waits for the occupancy grid and does not advertise its route-planning service.

Check:

```
rostopic list | grep grid
rosservice list | grep breadcrumb
```

### C. Waypoint/configuration error

A small change in:

```
spar_node/config/guidance.yaml
```

can produce incorrect mission geometry while leaving the rest of the stack operational.

Check:

```
rosparam get /guidance/mission
rosparam get /guidance/flight
```

### D. Breadcrumb planning parameter

Examples:

```
allow_diagonals
any_angle
enforce_bounds
```

Incorrect values may visibly change planned routes.

Check:

```
rosparam get /breadcrumb
```

### E. Incorrect SPAR action name

Expected:

```
/spar/flight
```

Likely symptom: Guidance reaches the SPAR waiting stage but cannot proceed.

## 16. Suggested Troubleshooting Order

Follow the dependency chain:

```
1. Is ROS running?
2. Is /grid being published?
3. Is Breadcrumb running?
4. Is /breadcrumb/request_path available?
5. Is MAVROS connected?
6. Is a valid local pose available?
7. Is SPAR running?
8. Is /spar/flight available?
9. Is the aircraft armed?
10. Is OFFBOARD selected?
11. Is Guidance progressing through mission states?
```

## 17. Key File Summary

| File | Role |
|---|---|
| `catkin_ws_G6/launch/control.launch` | Starts lower-level vehicle/control infrastructure |
| `spar/spar_node/launch/guidance.launch` | Starts Breadcrumb + Guidance |
| `spar/spar_node/scripts/guidance` | Main autonomous mission/state-machine logic |
| `spar/spar_node/config/guidance.yaml` | Mission, flight, topic and navigation configuration |
| `spar/spar_node/launch/spar.launch` | Starts SPAR motion layer |
| `breadcrumb/launch/breadcrumb.launch` | Breadcrumb planner configuration |
| `breadcrumb/src/breadcrumb/breadcrumb.cpp` | Occupancy-grid handling, service response and path formatting |
| `breadcrumb/src/breadcrumb/AStar.cpp` | A*/Theta* path-search logic |
| `qutas_lab_450/launch/environment.launch` | OptiTrack + occupancy-grid environment |
| `qutas_lab_450/launch/map_layout.yaml` | Occupancy-grid dimensions and obstacle layout |

## 18. Overall Summary

The AUT subsystem now follows a layered architecture:

```
Environment
    ↓
Breadcrumb route planning
    ↓
Guidance mission control
    ↓
SPAR motion execution
    ↓
MAVROS
    ↓
Pixhawk
```

The most important separation is:

```
Guidance   = mission decisions
Breadcrumb = safe route generation
SPAR       = movement execution
MAVROS     = flight-controller communication
```

The design preserves manual pilot authority, uses explicit result handling, prevents unsafe direct-flight fallback when planning fails, and centralises mission configuration in `guidance.yaml`.

For troubleshooting, the quickest approach is to follow the dependency chain from `/grid` and Breadcrumb through Guidance, SPAR, MAVROS and finally the aircraft state.